

Summary: Functions in Python

Functions are **reusable blocks of code** designed to perform specific tasks, allowing you to organise code into manageable "chunks" for better readability, debugging, and maintenance. They promote code reusability and help developers avoid repetitive tasks.

1. Types of Functions

The sources categorise functions into two main types:

- **User-defined functions:** Created by a programmer to solve specific problems or encapsulate specific functionality. These are defined using the **def** keyword.
- **Built-in functions:** Pre-defined functions bundled with Python (e.g., `print()`, `len()`, `max()`, `sum()`). These are generally **faster and more efficient** because they are pre-existing and optimised for performance.

2. Defining and Calling Functions

To define a function, use the following syntax:

- **def:** The keyword that signals a function definition.
- **Function Name:** A name following Python's naming conventions.
- **Parameters:** Optional placeholders for input data, defined within parentheses `()`.
- **Colon (:):** Denotes the start of the function body.
- **Indentation:** Statements inside the function must be indented.

Calling a function is done by using the function name followed by **arguments** in parentheses: `function_name(arguments)`.

3. Parameters and Arguments

- **Parameters** are the **placeholders** specified in the function definition.
- **Arguments** are the **actual values** supplied to the function when it is called.

Special Argument Handling

- **Required Arguments:** Functions must be called with the correct number of arguments; otherwise, a **TypeError** occurs.
- **Arbitrary Arguments (*args):** Adding a ***** before a parameter allows the function to accept any number of arguments, which are received as a **tuple**.
- **Keyword Arguments:** Arguments can be passed using the `key = value` syntax, meaning the **order of the arguments does not matter**.
- **Default Parameters:** You can assign a default value to a parameter (e.g., `def my_func(country = "Norway"):`). If the function is called without an argument, the default is used.

4. The Return Statement

The `return` statement is used to exit a function and pass a value back to the caller.

- **Default Return:** If a `return` statement is omitted, the function returns `None` by default.
- **Multiple Values:** Functions can return multiple values separated by commas, which Python implicitly packages as a **tuple**.
- **Placement:** Using `return` outside of a function body will raise a `SyntaxError`.

`print()` VS. `return`

The sources distinguish between these two based on their purpose:

- `print()` is used to **display** or output something to the console.
- `return` is used to **give back a value** that the function processed so it can be used elsewhere in the program.

5. Scope and Namespaces

Scope refers to the region where a variable is visible.

- **Global Scope:** Variables defined outside any function, accessible from anywhere.
- **Local Scope:** Variables defined inside a function, accessible only within that function.
- **global keyword:** Used to create or modify a global variable from within a function's local scope.

The LEGB Rule

A **namespace** is a container (like a box) where Python stores names and their objects. Python follows the **LEGB rule** to find a name's value in this specific order:

1. **L (Local):** The current function's box.
2. **E (Enclosing):** The box of any enclosing (outer) functions if nested.
3. **G (Global):** The script's top-level box.
4. **B (Built-in):** The box containing Python's built-in names.



Notes and Tips

- **Shadowing:** If you create a variable inside a function with the same name as a global variable, the local one "shadows" the global one—the function uses the local value, while the global variable remains unchanged.
- **Parameter Order:** In definitions, all parameters **without** default values must come **before** those **with** default values to avoid a `SyntaxError`.

- **Higher-Order Functions:** You can pass a function as an argument to another function (e.g., a `transform_list` function that takes a `double` function as an argument).
- **Empty Functions:** If you define a function with no content, you must use the `pass` statement to avoid an error.
- **Nested Execution:** When functions are nested, the **outer function is processed first**, but it only completes its execution after the inner function has finished.

Analogy: Think of a function like a **recipe**. The **parameters** are the ingredients listed in the recipe book, while the **arguments** are the actual ingredients you take from your fridge when you start cooking. Using `print()` is like showing a friend the cake you made; they can see it, but they can't take it home. Using `return` is like actually giving them the cake so they can eat it or save it for later.